

从已验证能力到合法派生

面向 Agent 的预研开发形式方法

(教材式草案 · 持续修订)

ascendc 工程 · 专题分支 `research/formal-lang-dag`

摘要

愿景：把「层层递进、先验证底层再拼高层」的工程直觉，提升为一套可教学、可门禁、可对照实验的形式方法——使 Agent（以及人类协作者）在复杂预研任务中，**只沿已验证能力及其合法组合**生长实现，从机制上压缩跳层发明与幻觉空间；最终服务一般预研开发，而非仅服务某一密码标准的算子清单。

写法：仿教材而非论文——先从特例讲故事，再抽象数学对象，再复盘与前瞻；证明与完备性后置，允许边做边补。**地位：**`docs/research/` 调研草稿，非定稿、非交付验收依据。

压力测试床：ML-KEM-1024 链上的 `pke.keygen / kem.keygen / kem.encaps`，并以 `kem.decaps` 为理论前瞻试金石；`*-correctness-*` 作为「只求正确、不计优化」的对照组标本。

目录

1 愿景、读者与本文怎么读	2
1.1 愿景（一句话）	2
1.2 我们希望最终得到什么	2
1.3 研究路线：从特殊到一般	2
1.4 阅读地图（教材顺序）	2
2 一个完整例子：我们怎样做出 ML-KEM-1024 的 KEM.KeyGen	2
2.1 我们要交付什么	3
2.2 FIPS 203 原文：三层算法怎样套在一起	3
2.3 动手之前，我们手里已经有什么	5
2.4 我们怎样探出来：工程路径与数据依赖	6
2.5 从这个例子出发，我们应当问什么？	7
2.6 本工程现在能回答这四组问题吗？	8
3 回溯：一层层剥开直到平台事实	9
3.1 分层语言（先约定叫法）	9
3.2 剥洋葱示意（KeyGen 链，叙述级）	9
3.3 为何必须剥到 P0	10
3.4 与现有仓库机制的对应（尚未形式化前的直觉）	10

4 从故事到对象：DAG、语法树与过程	10
4.1 三个容易混的东西，先拆开	10
4.2 节点（已验证能力）	11
4.3 边的三种类型	11
4.4 证书状态（含非单调）	11
4.5 形式语言视角（草案）	11
4.6 自动机视角（草案）	12
4.7 立论（工作定义）	12
5 门禁与工作流：Agent 被允许做什么	12
5.1 合法性规则（v0）	12
5.2 四步工作流	12
5.3 域无关性	12
6 复盘：理论如何对照 kem.keygen 与 kem.encaps	13
6.1 复盘的目的	13
6.2 kem.keygen 校准问题单	13
6.3 kem.encaps 复盘要点（讨论共识）	13
6.4 期望产出的「具象化对照表」（后续填空）	13
7 前瞻：用理论指导尚未完成的 kem.decaps	13
7.1 为什么 Decaps 适合做试金石	13
7.2 前瞻时 Agent（或人类）应交付的最小闭包	14
7.3 成功标准（阶段性）	14
8 对照组：correctness 标本的意义	14
8.1 为何保留 correctness	14
8.2 在方法论中的角色	14
8.3 建议比较轴	14
9 未决议事项、日程与维护	15
9.1 刻意未决议	15
9.2 近期填空顺序（与「特殊 → 一般」一致）	15
9.3 文档与分支约定	15
9.4 编译	15

1 愿景、读者与本文怎么读

1.1 愿景（一句话）

我们想表达的是：预研开发**不应当**变成「对着算法说明书自由发挥写核」。我们应当只在**已经拿到证书**的能力之上，按规则组合出更复杂的任务；静态 DAG 只是「引理库谁依赖谁」的一张投影图，真正的作业过程还要另说。

1.2 我们希望最终得到什么

1. 一套**教材式叙述**：新人 / 新 Agent 能从特例读懂「图从哪来、树怎么长」；
2. 一套**可操作门禁**：任务启动必须先交依赖闭包与缺项，再写码；
3. 一套**对照实验协议**：方法论指导下的实现 vs correctness 标本 vs 人工指导路径；
4. (成型后) 可解释的接受/拒绝轨迹，以及必要的证明——**不作为**本稿前置条件。

1.3 研究路线：从特殊到一般

阶段	做什么	刻意后置
特例引出	工程故事 → 图与语法树	完备公式
套公式	用数学 复述 已见现象	证明
改公式	缺口反过来改理论	为漂亮删工程事实
套别例	Encaps 复盘；换域；Decaps 前瞻	一次塞太多缺口
迭代方法论	门禁与流程收敛	急进 Rule/Skill
成型后	证明、可解释性	过早锁死抽象

1.4 阅读地图（教材顺序）

1. 第 2 章：**完整例子**（KEM.KeyGen：算法原文、我们有什么、怎样做出来、引出课题）；
2. 第 3 章：**剥洋葱**（从 PKE 继续往下到平台事实）；
3. 第 4-5 章：**抽象与门禁**（理论草案）；
4. 第 6-7 章：**复盘与前瞻**（用理论看实现）；
5. 第 8 章：**correctness 对照组**；
6. 第 9 章：未决议事项与维护约定。

讨论过程的流水账见同目录 形式语言 DAG 预研讨论纪要.md（单文件持续刷新）。

2 一个完整例子：我们怎样做出 ML-KEM-1024 的 KEM.KeyGen

本章用一个**已经做完**的真实任务当例子。我们先说清楚「要算什么」，再贴 FIPS 203 里的算法原文，然后交代我们手里已经有什么、前期做过哪些实验、最后怎样拼出 KEM.KeyGen。例子讲完之后，我们再**提问**——这些问题应当能把读者自然带到后文的研究课题里。

2.1 我们要交付什么

我们的目标是：在昇腾 AI Core 上用 AscendC 实现 **FIPS 203 ML-KEM-1024** 的密钥生成 (ML-KEM.KeyGen, 即 Algorithm 19)。

读者可以把交付物理解成下面这张黑盒表 (细节以 stable 目录 customspect 为准)：

方向	内容	说明
输入	seed_d.bin (4B)	Host 写入；设备侧派生 d 、 z
输入	NTT LUT (静态)	与 PKE KeyGen 同表
输出	ek_kem.bin (1568 B)	与 ek_PKE 相同
输出	dk_kem.bin (3168 B)	与 liboqs 展开布局一致

我们不把 d 、 z 、中间 $\hat{A}$ 、 $\hat{s}$ 等当作交付文件落盘。验收方式是：run.sh 跑 CPU 孪生 + SIM，输出与 golden / liboqs 对拍一致。

2.2 FIPS 203 原文：三层算法怎样套在一起

标准把 KEM 密钥生成拆成三层。下面给出与 **FIPS 203 原文一致的参数、Input/Output 与步骤编号**。正文引用 NIST FIPS 203 Algorithm 13 / 16 / 19；本文实例固定为 ml_kem_1024 ($k=4$ 时 $|ek|=1568$ B, $|dk|=3168$ B)。

Algorithm 19 ML-KEM.KeyGen() (FIPS 203 §7.1)

参数集 (ml_kem_1024): $n=256$, $q=3329$, $k=4$, $\eta_1=\eta_2=2$, $d_u=11$, $d_v=5$ 。

Input: (无显式输入； d 、 z 在算法内由 RBG 采样，见步骤 1-2。)

Output: 封装密钥 $ek \in \mathbb{B}^{384k+32}$ ；

解封装密钥 $dk \in \mathbb{B}^{768k+96}$ 。

1: $d \leftarrow \text{\$B}^{32}$

2: $z \leftarrow \text{\$B}^{32}$

3: if $d = \text{NULL}$ or $z = \text{NULL}$ then

4: return $\perp$

RBG 失败指示

5: end if

6: $(ek, dk) \leftarrow \text{ML-KEM.KeyGen_internal}(d, z)$

7: return (ek, dk)

Algorithm 16 ML-KEM.KeyGen_internal(d, z) (FIPS 203 §6.1)

参数集 (ml_kem_1024): $n=256$, $q=3329$, $k=4$, $\eta_1=\eta_2=2$, $d_u=11$, $d_v=5$ 。

Input: 随机种子 $d \in \mathbb{B}^{32}$ ；随机种子 $z \in \mathbb{B}^{32}$ 。

Output: 封装密钥 $ek \in \mathbb{B}^{384k+32}$ ；

解封装密钥 $dk \in \mathbb{B}^{768k+96}$ 。

1: $(ek_{\text{PKE}}, dk_{\text{PKE}}) \leftarrow \text{K-PKE.KeyGen}(d)$

2: $ek \leftarrow ek_{\text{PKE}}$

3: $dk \leftarrow dk_{\text{PKE}} \| ek \| H(ek) \| z$

4: return (ek, dk)

其中 $H = \text{SHA3-256}$ (FIPS 203 记号 H)。

Algorithm 13 $K\text{-PKE.KeyGen}(d)$ (FIPS 203 §5.1)

参数集 ($m1_kem_1024$): $n=256$, $q=3329$, $k=4$, $\eta_1=\eta_2=2$, $d_u=11$, $d_v=5$ 。

Input: 随机种子 $d \in \mathbb{B}^{32}$ 。

Output: 加密密钥 $ek_{\text{PKE}} \in \mathbb{B}^{384k+32}$;

解密密钥 $dk_{\text{PKE}} \in \mathbb{B}^{384k}$ 。

```

1:  $(\rho, \sigma) \leftarrow G(d\|k)$  域分离: 字节 33 为模块维  $k$ 
2:  $N \leftarrow 0$ 
3: for  $i \leftarrow 0$  to  $k-1$  do
4:   for  $j \leftarrow 0$  to  $k-1$  do
5:      $\hat{A}[i, j] \leftarrow \text{SampleNTT}(\rho\|j\|i)$ 
6:   end for
7: end for
8: for  $i \leftarrow 0$  to  $k-1$  do
9:    $s[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N)); N \leftarrow N + 1$ 
10: end for
11: for  $i \leftarrow 0$  to  $k-1$  do
12:    $e[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N)); N \leftarrow N + 1$ 
13: end for
14:  $\hat{s} \leftarrow \text{NTT}(s)$ 
15:  $\hat{e} \leftarrow \text{NTT}(e)$ 
16:  $\hat{t} \leftarrow \hat{A} \circ \hat{s} + \hat{e}$  NTT 域矩阵-向量乘加
17:  $ek_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\hat{t})\|\rho$ 
18:  $dk_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\hat{s})$ 
19: return  $(ek_{\text{PKE}}, dk_{\text{PKE}})$ 

```

读者只需抓住一条结构链: $\text{Alg.19} \Rightarrow \text{Alg.16} \Rightarrow \text{Alg.13}$ 。Alg.19 比 Alg.13 多出来的主要是步骤 1-2 的 z 、Alg.16 步骤 3 的拼接——并不是把 NTT 或矩阵乘从头再写一遍。

本仓工程差异 (验收已锁定, 写在算法旁便于对照)

- 我们可复现跑分时, Host 只提供 4 B `seed_d`; d 与 z 在设备 UB 内派生 (对应 Alg.19 步骤 1-2 的确定性扩展)。
- 我们对拍 liboqs 时, dk 为 3168 B 展开式 $dk_pke\|ek\|H\|z$, 与 FIPS 代数最小形态一致, 但字节布局与偏移以 `customspec` 为准。

2.3 动手之前，我们手里已经有什么

到做 *KEM.KeyGen* 之前，本仓库并不是从一张白纸开始。我们已经在 AscendC 上做过大量单点与分段实验，并且多数有 CPU+SIM 对拍记录。下面这张**资产清单**把「当时手里有什么」写清楚；文字说明与表格对照阅读。

四类家底（文字）

1. **平台层 (P0)**：我们验证过 DataCopy、TPipe/TQue、MIX 核同步、MMAD tiling 等。
2. **密码原语层 (P1)**：我们验证过 NTT (poly-batch)、basemul、SHAKE/SHA3、CBD、ByteEncode₁₂、SampleNTT 等。
3. **PKE 全链 (P2)**：我们已有交付级 stable PKE KeyGen (2 launch, KAT 通过)。
4. **对照标本**：我们保留 **-correctness-**，只求可执行与字节正确，不计优化。

资产清单（表格；锚点以活跃 INDEX 为准）

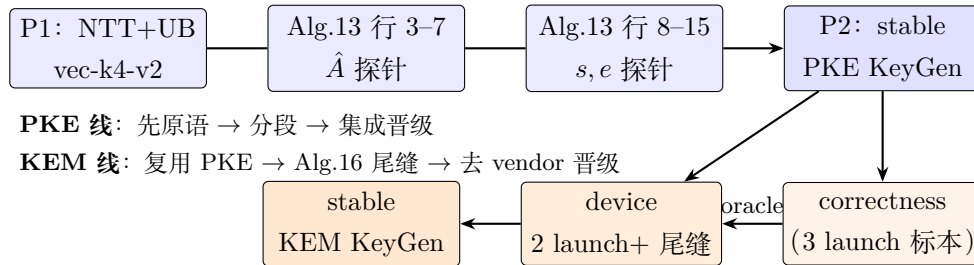
层	能力（简述）	仓库锚点（示例）	证书	备注
P0	DataCopy / 搬运	docs/notes/ascendc-DataCopy ...	文档 + 探针	平台知识库
P0	MMAD + tiling	pass-int8-matmul-cube-...等	CPU+SIM	闭合条件见 notes
P0	TPipe / 细同步	KeyGen prep 技术总结	CPU+SIM	翻车高发区
P1	NTT 三段式 poly-batch	pass-fix-...-vec-k4-v2	CPU+SIM	ML-KEM 活跃基线
P1	SampleNTT / Alg.7	pass-fix-f203-alg7-sample-ntt-cpu	CPU+SIM	
P1	CBD $\eta=2$	pass-fix-f203-alg8-cbd-eta2-k4	CPU+SIM	
P1	ByteEncode ₁₂	pass-fix-... -byteencode12-vec-k4	CPU+SIM	
P2	Alg.13 行 3-7 $\hat{A}$	pass-fix-f203-alg13-lines3-7-cpu	CPU+SIM	≈542k tick KEM 直接依赖
P2	Alg.13 行 8-15 s, e	pass-fix-f203-alg13-lines8-15-cpu	CPU+SIM	
P2	Alg.13 device 全链	pass-fix-f203-alg13-device-keygen-k4	CPU+SIM	
P2	stable PKE KeyGen	stable-fips203-mlkem-pke-keygen-k4	交付 KAT	
P3	KEM KeyGen correctness	fix-f203-alg19-kem-keygen-correctness-k4	CPU+SIM	对照标本
P3	KEM KeyGen device	pass-fix-f203-alg19-kem-keygen-device-k4	CPU+SIM	≈713k tick
P3	stable KEM KeyGen	stable-fips203-mlkem-kem-keygen-k4	交付 KAT	本例终点

当我们决定做 *KEM.KeyGen* 时，我们并不是让 Agent 从 FIPS 伪代码一行行翻译成 AscendC，而是心里已经有上表这张「哪些积木是绿的」的清单。

2.4 我们怎样探出来：工程路径与数据依赖

这张清单不是一天长出来的。本节用两张图把同一件事说清楚：**第一张**是仓库时间线（先长 PKE，再接 KEM 尾缝）；**第二张**是 FIPS Alg.19→16→13 的数据依赖（与 §2.2 步骤编号对照）。

图一：工程探路（PKE 线 → KEM 线）



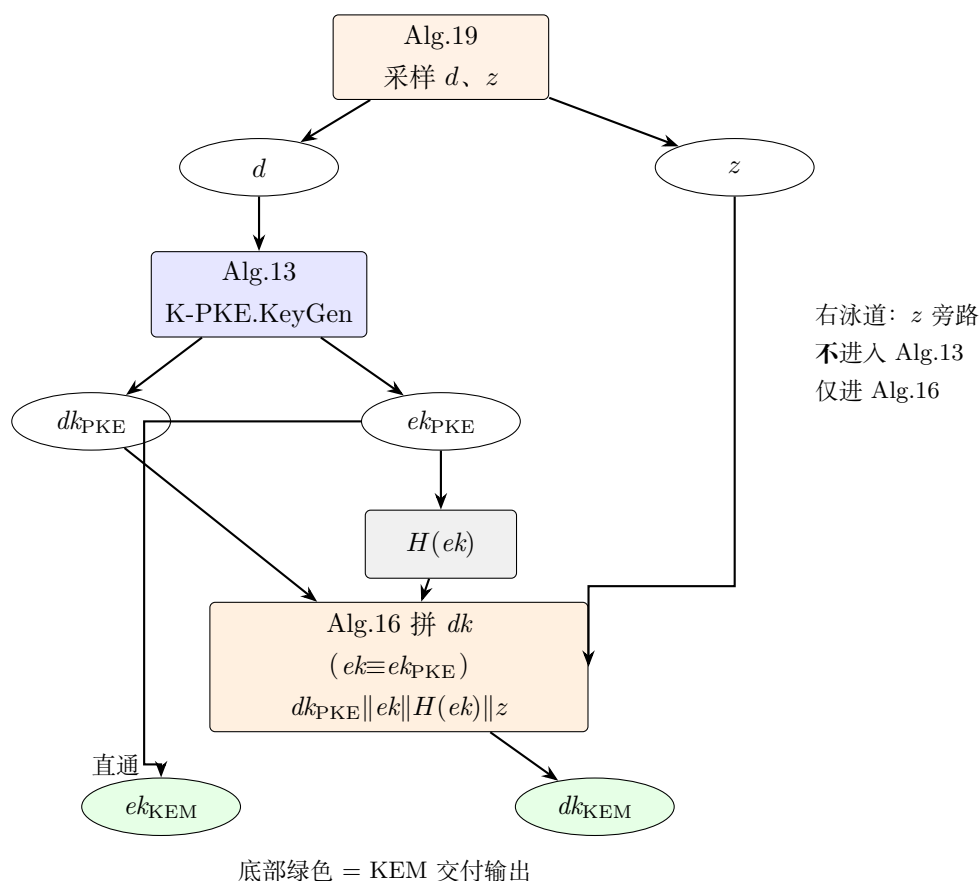
- **PKE 线：**我们先把 NTT、内积等做成向量探针；再把 Alg.13 拆成 $\hat{A}$ 、 s, e 、device 全链，逐段 PASS；最后集成晋级 stable。
- **KEM 线：**我们语义上调用 Alg.13（步骤 1 即 K-PKE.KeyGen）；设备上在 Launch-2 尾部追加 Alg.16 步骤 2-3（ H 、拼 dk ）；编排上保持 2-launch；验收上 correctness 作 oracle，device 去 vendor 后晋级 stable。

设备侧 Launch 与 FIPS 行的对应（本仓实现）

Launch	设备段	FIPS 行	要点
L1	f203_keygen_prep	Alg.13 步骤 1-15	双 AIV； $\hat{A}$ 、 $\hat{s}$ 、 $\hat{e}$
L2	mmad_custom + 尾缝	Alg.13 步骤 16-21	NTT、内积、ByteEncode
L2 尾	KemKgTailFused	Alg.16 步骤 2-3	z 、 $H(ek)$ 、写 dk

若用一句话概括这次成功：**我们把 FIPS 的三层算法，映射成「已证 PKE 链 + 一条单独想清楚的 KEM 尾缝」**，而不是在 KEM 任务里重写 NTT。

图二：FIPS 三层的数据依赖（图感） 图一讲「仓库里怎样长出来」；图二只看标准数据流——重计算在 Alg.13， z 不进 Alg.13，只进 Alg.16 的 dk 拼接； ek_{KEM} 对 ek_{PKE} 是直通。



读图时抓住三点：

1. **PKE 主体**：几乎全部重计算落在 Alg.13； d 进、 ek_{PKE}/dk_{PKE} 出。
2. **KEM 尾缝**：Alg.16 把已有片段拼成 dk_{KEM} ； ek_{KEM} 对 ek_{PKE} 直通（图底左侧绿叶），不重做 NTT。
3. **汇点而非树**：NTT、CBD、MMAD 等还不画进本图——它们也不专属于 KeyGen，Encrypt / Encaps 会复用；因此本图只是更大有向无环图里的一个汇点，不是一棵只往下长的树。

2.5 从这个例子出发，我们应当问什么？

例子讲完，真正重要的是**问题**。如果我们的研究课题是「怎样让 Agent 预研少幻觉、少撞墙」，那么面对 KEM.KeyGen 这个故事，我们至少应当认真问下面四组问题。

第一组：标准与工程之间的缝隙

1. FIPS 只告诉我们算什么；它并没有告诉我们，在熵上应当先验哪几块、按什么顺序拼。我们凭什么在动手前就说「可以直接站在 stable PKE 上」——凭的是目录名，还是某种可写的证书？
2. 探针列表 (INDEX) 像一张「能力菜单」。菜单本身是否等于合法拼装说明书？若不等，缺的那一页应当长什么样？

第二组：「单点都对」之后，组合还对吗

3. 当 NTT 探针 PASS、PKE KeyGen PASS 之后，**谁**来保证「2-launch 接缝」「Alg.16 尾拼 *dk*」也一定 PASS？我们能否把这种「组合不免费」写进理论，而不是每次靠工程师的经验？
4. 若我们把 correctness 路线（更多 launch、更保守同步）与 device/stable 路线（更少 launch、吃到优化）都称为「正确」，**我们究竟在验收什么**——仅仅是输出字节，还是也要验收「沿已证能力生长」？

第三组：Agent 若只知道算法和菜单，会发生什么

5. 若我们只给 Agent 一段 Alg.19 伪代码 + INDEX 里有哪些探针 PASS，**但不给**确定的拼装路线，搜索空间会有多大？我们亲眼见过：过程高度混乱、大量回退、极耗 token，最后只留下「能跑对」的 correctness 标本。
6. 我们能否把搜索空间收束为「只允许沿已证书边生长」，同时又**不把**工程优化堵死？收束的规则应当写在图里，还是写在某种「合法句子」里？

第四组：怎样把一次成功变成可复用的方法

7. 这次 KEM.KeyGen 的成功，能否被复述成一张**闭包表**：依赖谁、缺了谁、哪条边有证书、哪条边被禁止？若下一个人（或 Agent）做 Encaps / Decaps，我们能否**先交这张表再写码**？
8. 当上游改了 tiling 或同步壳，下游哪些节点应当自动标「脏」？我们是否需要一种比「记得上次 PASS」更可靠的失效传播？

2.6 本工程现在能回答这四组问题吗？

四组问题都很好，但「能回答」要分层次：工程实践里我们已有碎片，形式化方法论里我们尚未闭环。下表诚实对照（2026-07 专题讨论时的判断）。

问题组	本工程 已能 说明的	仍缺的	判断
第一组	INDEX、customspec、baseline-registry 已在约束引用来源	机器可读的「证书对象」；菜单 ≠ 拼装说明书的显式规则	部分能答
第二组	我们有 seam 探针/集成探针实践；correctness vs device 在 customspec 有对照	「组合不免费」未写成统一理论；验收口径仍以 I/O 为主	部分能答
第三组	correctness 历史是 反面教材 （高成本、不可控）	正向的「合法派生」门禁尚未工具化；Agent 仍可能绕开	能答负例；正例在建
第四组	单任务可手写 INTEGRATION_PLAN、闭包直觉	统一闭包表 schema；dirty 传播无自动机制	部分能答

逐组一句话

- **第一组**：我们能指着 stable PKE 说「KEM 应站在这里」，但还缺一张人人（含 Agent）都认的证书页。
- **第二组**：我们知道接缝要单独验（例如 2-launch、尾缝），但还没把这条经验收成可检查的规则。
- **第三组**：我们能用 correctness 证明「只给菜单会爆炸」；方法论要证明的是「给规则后能收敛」——这正是本专题要做的事。
- **第四组**：我们能复盘 KEM.KeyGen 的故事，但 Encaps/Decaps 还不能先交闭包表再写码作为硬门禁。

因此：本工程能支撑这四组问题的提出与部分实证，但不能宣称方法论已经成立。后文的工作，就是把表中「仍缺的」一列逐项补上，并用 Encaps / Decaps 做检验。

这些问题的共同指向 上面这些问题，表面上各不相同，但它们指向同一条研究主线：

我们能否把「已验证能力 + 合法组合 + 动态证书」整理成一套可教、可检查、可对照实验的形式方法——

让 Agent 的工作从「在菜单里瞎拼」变成「在自动机上走合法派生」？

后文将先继续剥洋葱（看清 PKE 之下还有哪些底层事实），再把这些直觉收成 DAG、形式语言与配置自动机的语言（第 4 章起），并用 Encaps 复盘、Decaps 前瞻来检验这套语言是否管用。

3 回溯：一层层剥开直到平台事实

3.1 分层语言（先约定叫法）

层	含义（本仓库语境）
P3	顶层算子 / 算法入口（如 KEM.KeyGen、KEM.Encaps）
P2	算法构件（如 PKE.KeyGen / Encrypt / Decrypt 全链）
P1	领域原语（NTT、basemul、CBD、ByteEncode、SHAKE...）
P0	平台公理（DataCopy、向量算子、MMAD+tiling、TPipe/TQue 同步壳...）

注意：P0-P3 是方法论分层，不等于目录树一一对应；一个目录可覆盖多层接缝。

3.2 剥洋葱示意（KeyGen 链，叙述级）

从引子向下，逻辑顺序大致是：

```
kem.keygen (P3)
|- seam: seed->(d,z), dk expand, production I/O
~- pke.keygen (P2, Alg.13)
  |- seam: prep || compute (e.g. 2-launch)
  |- a_hat / SampleNTT 相关 (P1..)
  |- s,e sample + CBD (P1..)
```

```
|~ NTT + matmul/basemul + ByteEncode... (P1)
~ 以上一切最终落到 (P0):
    DataCopy / UB 布局 / MMAD tiling / PIPE · TQue 同步
```

每一层在教材叙述里建议用同一模板四问（避免写成探针编年史）：

- 1. **I/O**：黑盒输入输出是什么？
- 2. **依赖**：必须已经成立的下层能力是谁？
- 3. **证书**：CPU+SIM（及交付 KAT）锚在哪个目录 / 哪次验收？
- 4. **接缝**：本层相对下层多出来的组合点是什么？是否单独验收过？

3.3 为何必须剥到 P0

高层「NTT 已对」并不蕴含「在新的 launch 编排与同步壳下仍然对」。仓库历史反复表明：翻车常发生在**搬运与同步接缝**，而非代数公式本身。因此 P0 不是「太底层不必写进方法论」，而是**终端事实**的主要来源之一。

3.4 与现有仓库机制的对应（尚未形式化前的直觉）

- 活跃 ascendc-tests/INDEX.md / examples/*/INDEX.md：当前可引用的能力菜单；
- frozen/ + FROZEN.md：否定边（可读关闭原因，禁止抄实现）；
- baseline-registry：golden 内核允许调用的已验证计算块；
- customspec：examples 写码的规格门禁。

这些机制已经在强制「不要随便发明」；本稿要把它们收成**统一的图 + 派生语言**。

4 从故事到对象：DAG、语法树与过程

4.1 三个容易混的东西，先拆开

概念	回答的问题	不回答的问题
能力 DAG	引理库里谁依赖谁？	某次任务如何一步步展开？
语法树 / 派生	这次任务的展开句子是什么？	全局所有共享边的全貌？
配置自动机	证书如何增减？何时接受/拒绝？	代数是否符合 FIPS？

DAG 是库的投影；语法树是**一次作业**的展开；自动机是**作业过程**的状态演化。三者一起，才接近「预研怎么合法地做完」。

4.2 节点（已验证能力）

节点 v 建议携带：

- `id`：稳定名（如 `pke.keygen.k4`）；
- `layer`：P0–P3；
- `iface`：I/O、dtype、形状/布局、同步约定——**可复用的是 `iface`，不是源码**；
- `cert`：路径 + CPU+SIM (+KAT) + 版本锚点；
- `forbid`：否定约束；
- `repo`：权威目录锚点。

4.3 边的三种类型

1. **语义依赖**： u 的正确性需要 v 的 `iface`（如 $\text{NTT} \rightarrow \text{PKE.KeyGen}$ ）；
2. **接缝（compose）**： A 、 B 各自有证， $A \circ B$ 仍须单独成节点获证（「组合不免费」）；
3. **否定边**：显式禁止（frozen 模板、未登记计算块、Host 默认密码路径等）。

4.4 证书状态（含非单调）

$\text{unproven} \rightarrow \text{probe} \rightarrow \text{lemma} \rightarrow \text{deliverable}$ ，以及 `dirty`。

`dirty` 表示上游 `iface/tiling` 变更后下游证书失效——引理库 Γ **可收缩**。这与「证明只增不减」的古典图景不同，却符合工程真实。

4.5 形式语言视角（草案）

字母表 Σ ：已认证叶子 `iface` 的「调用」+ 有限组合子（分核、分 `launch`、`workspace`、`host` 调度...）。

- 字 $w \in \Sigma^*$ ：一条实现计划 / 展开轨迹；
- 语言 L ：全体合法计划；
- 产生式：目标改写 $T \Rightarrow u_1 \cdots u_k$ （当且仅当依赖与接缝已声明）；
- 成员判定：对 Agent 提交的闭包表做合法派生检查；
- 禁字：frozen / forbid / 未登记块。

L 未必正则：嵌套与属性约束更接近上下文无关、属性文法，或判断式 $\Gamma \vdash T$ 。

4.6 自动机视角（草案）

配置 $q = (\Gamma, F, S)$ ：已证书集、前沿目标、接缝与约束。转移包括：Expand、Probe、Certify、Compose、Dirty、Freeze。

- **接受**：目标 $T^* \in \Gamma$ 且生产 I/O 契约满足；
- **拒绝**：未证书边、frozen 边、跳层发明终端。

4.7 立论（工作定义）

预研开发的合法性 $\approx$ 在已证书能力生成的语言 L 上，由配置自动机搜索接受路径；

幻觉 $\approx$ 非法转移 / 无证书边。

FIPS/liboqs 等仍是语义 oracle——形式方法管「从哪砌砖」，不管「砖的代数是否被重新发明」。

5 门禁与工作流：Agent 被允许做什么

5.1 合法性规则（v0）

接到目标 T 时，只允许：

1. 展开依赖闭包（语义边 + 接缝边）；
2. 分类：已有 lemma/deliverable；缺项；仅否定边；
3. 缺项 $\Rightarrow$ 先探针/exp 并获证，再引用；禁止在 T 任务里顺便发明 P0/P1；
4. 新接缝 = 新节点，须认证；
5. 验收看 I/O 与 golden/KAT，不看「与参考源码同构」；
6. 自包含：挂 iface，不跨目录偷源码（仓库既有例外除外）；图边 $\neq$ #include。

5.2 四步工作流

1. 目标声明 T、I/O、验收级别（probe / lemma / deliverable）
2. 闭包展开 P0--P3；ok / missing / dirty；接缝清单
3. 最小补洞 只补 missing 与新接缝；forbid 写入否定边
4. 闭环写回 新节点入图；失败则改规则或 Freeze，不默认可复用

先交闭包表，再写码。 customspec / STATUS / registry 是文书形态，不是替代闭包表。

5.3 域无关性

抽出密码词之后，对象仍是：终端 (axiom)、非终端 (goal)、产生式 (compose)、证书 (judgment)。同一套问题适用于非密码 AscendC 预研；ML-KEM 只是 oracle 清晰、接缝丰富的**压力测试床**。

6 复盘：理论如何对照 *kem.keygen* 与 *kem.encaps*

6.1 复盘的目的

不是证明「我们早就在按理论做」（事后贴标签无价值），而是：

1. 检查分层与接缝是否可被理论语言说清；
2. 找出偏离（例如曾依赖 *correctness vendor* / *frozen* 模板）并记为**方法论债**或**否定边**；
3. 总结「数学原理如何被具象化为目录、*launch*、证书」。

6.2 *kem.keygen* 校准问题单

- P3 是否只依赖已证书 P2，而非在 KeyGen 任务内重写 NTT？
- 2-launch、Alg.16 尾、*dk* 展开等接缝是否显式？
- *baseline-registry* 中的 *golden* 块是否都在图上有 lemma？
- 否定边是否写入（Host 默认禁密码、禁把 *liboqs* 当 AscendC 规格）？

仓库锚点(查阅用):[examples/stable/stable-fips203-mlkem-kem-keygen-k4/](#),[docs/specs/fips203-mlkem1](#) 及相关 *pass-fix-* device* 探针。

6.3 *kem.encaps* 复盘要点（讨论共识）

- 合法路径应挂已证 *pke.encrypt* 与哈希/G 等 lemma，而不是以 *correctness* 内 *vendor* 为实现模板；
- 与 *stable Encrypt* 的编译期/接缝关系必须写进闭包，而不是「觉得复用了」；
- 若实现史存在混乱探索，复盘文应诚实记录——那是对照组与方法论的动机，不是羞耻。

6.4 期望产出的「具象化对照表」（后续填空）

理论对象	工程具象	证书形态
节点 v	探针/ <i>exp/stable</i> 目录	STATUS + <i>run.sh</i> 对拍
语义边	「必须先有某某能力」	INDEX / <i>customspec</i> 引用
接缝节点	多 <i>launch</i> / 融合边界	集成探针单独 PASS
否定边	<i>frozen</i> / <i>forbid</i> 条文	FROZEN.md、Rule
字 $w \in L$	一次任务的闭包表 + 计划	评审通过再写码

7 前瞻：用理论指导尚未完成的 *kem.decaps*

7.1 为什么 Decaps 适合做试金石

Decaps 同时依赖 *Encrypt*、*Decrypt*、KeyGen I/O 与 FO 失败路径，是典型的**多父 DAG**；且若理论只能解释已完成故事、不能生成待做清单，则方法论停在修辞。

7.2 前瞻时 Agent（或人类）应交付的最小闭包

1. 目标 I/O 与验收级别；
2. 依赖节点清单（标 ok / missing / dirty）；
3. 接缝清单（含对称失败 / 重加密比对等是否独立成节点）；
4. 否定边（不可引用的 correctness vendor / frozen 实现）；
5. 最小补洞顺序（先探针哪条缝）。

7.3 成功标准（阶段性）

- **弱成功**：闭包表可执行；偏差归因到缺边/假边/脏证书，并回写本章理论；
- **强成功**：在门禁下完成 device/stable 级实现，且过程指标优于 correctness 探索史（见下一章比较轴）。

8 对照组：correctness 标本的意义

8.1 为何保留 correctness

-correctness- 在完全不考虑优化的前提下，只追求可执行性与结果正确：多 kernel launch、多搬运与同步——这是「一定能搭出正确积木」的保守拼法，性能不可用，也几乎吃不进后续 P0/P1 工程优化。

它还记录了一段真实的 Agent 过程史：故意不给确定性拼装路线时，探索成本极高（失败、卡死、回退；曾短期内耗尽大量 token），最终仅得到「结果正确」的代码。

8.2 在方法论中的角色

角色	含义
正确性锚	下界存在性：积木可搭、I/O 可对
结构反例	展示「只求对」会长成怎样的 launch/同步膨胀
过程对照	无确定路线的搜索成本
实验基线	与方法论 Agent、人工指导路径三方比较

纪律：correctness 是**基线标本**，不是活跃实现模板；可读、可对拍、可对比，**禁止**当抄码来源（与 frozen 出门不带码一致）。

8.3 建议比较轴

1. **过程**：token / 轮次 / 回退；是否先交闭包表；非法边是否被拦截；
2. **结构**：launch 数、同步点、lemma 复用、接缝是否显式；
3. **结果**：CPU+SIM；SIM tick 相对 correctness 的倍数。

三档：A 旧式无路线探索 $\sim$ correctness；B 新方法论 Agent；C 人工指导 device/stable。B 不必一次打赢 C；明显优于 A，即方法论第一关。

9 未决议事项、日程与维护

9.1 刻意未决议

- 节点是否 1:1 对应目录名；图的存储是 Markdown 表还是 YAML/JSON；
- 主叙事选工作流网 / Petri 网 / 属性文法 / 判断式中的哪一种；
- 何时写入 Rule/Skill；可判定性与复杂度是否单开理论线。

9.2 近期填空顺序（与「特殊 $\rightarrow$ 一般」一致）

1. 把第 2-3 章补成「带四问模板的具体洋葱图」（仍用特例语言）；
2. KeyGen 合法派生轨迹一页纸；
3. Encaps 理论轨迹 vs 实现轨迹对照表；
4. Decaps 前瞻闭包；
5. correctness 对照实验写成可执行协议；
6. 结论稳定后再考虑迁 docs/notes/。

9.3 文档与分支约定

- 本文件为专题**唯一** TeX/PDF 工作副本（无日期后缀），持续修订；
- 讨论纪要：形式语言 DAG 预研讨论纪要.md（同样单文件刷新）；
- 工作分支：research/formal-lang-dag；未经用户明确指令**不合人** main；
- 不记录题外的知识产权保护策略讨论。

9.4 编译

```
bash scripts/xelatex-clean.sh \  
docs/research/从已验证能力到合法派生-面向Agent预研的形式方法教材草案.tex
```